

CHAPTER 2: TYPESCRIPT

LEARNING OBJECTIVES

- Know the advantages of chosing TypeScript over JavaScript
- Be able to apply TypeScript's type system in applications
- Be able to develop TypeScript applications with functions and classes
- Understand the TypeScript type inference
- Be able to use TypeScript's operators to write compact code
- Understand what a Decorator is
- Be able to modularize a TS project with modules



2.1 OVERVIEW

WHAT IS TYPESCRIPT AND WHY USE IT?

TypeScript (TS) is a programming language developed by Microsoft. TS is a superset of JavaScript, i.e. it adds the following features to JS:

- static typing: just like in Java, everything needs to be statically typed
- interfaces: define structure of objects as contract between code parts
- classes and Object-Oriented Programming
- modules and namespaces: better code organization and reuse
- type inference: even if types are omitted, TS will infer them implicitly
- tooling and IDE support

Keep in mind: every valid JS code is also valid TS code!

WHY USE TYPESCRIPT INSTEAD OF JAVASCRIPT?

Main reason for TypeScript: it adds a **type system** to help you avoid many problems with dynamic types in JavaScript.

Example:

```
function getProduct(id){
  return {
    id: id,
    name: `Awesome Gadget ${id}`,
    price: 99.5
  }
}
// usage
const product = getProduct(1);
console.log(`The product ${product.Name} costs $${product.price}`);
```

Above code outputs "The product undefined costs $99.5", which is not quite what we expected!

Q: Can you spot the problem?

COMPILE TIME ERROR DISCOVERY WITH TYPESCRIPT

An improved version in TypeScript might look as follows:

```
type Product = { id: number, name: string, price: number };

function getProduct(id): Product {
  return {
    id: id,
    name: `Awesome Gadget ${id}`,
    price: 99.5
  }
}
// usage
const product = getProduct(1);
console.log(`The product ${product.Name} costs $${product.price}`);
```

Compiled with the TypeScript Compiler `tsc` we would get:
"12:36 - error TS2551: Property 'Name' does not exist on type 'Product'"

Also the TS linter in VS Code would indicate the problem while you type in your code.
In the next section we will explain everything you need to know about TS typing.

DIFFERENCE BETWEEN TYPE CHECKING IN JAVA AND TYPESCRIPT

Both languages have a static type system, but the way type checking works is different:

- Java uses a nominal type system, where compatibility is based on explicit type declarations. Even if two classes have the same attribute and method types, java considers them as different.
- TypeScript on the other hand uses a **structural type system**, meaning that two types are compatible if they have the same shape or structure (duck typing).

Example:

```
interface Person { name: string, age: number };
interface SomeoneWithaNameAndAnAge { name: string, age: number };

const v1: Person = { name: "Paula", age: 21 };
const v2: SomeoneWithaNameAndAnAge = { name: "Hugo", age: 31 };

function greet(p: Person): void {
  console.log(`Hi ${p.name}, i guess you are ${p.age}?`);
}

greet(v1); // output: Hi Paula, i guess you are 21?
greet(v2); // output: Hi Hugo, i guess you are 31?
```

The TS type inference mechanism compares the structure (in terms of properties) of the two types and determines that both types are compatible.

A Java compiler would produce an error message for the second call to greet()!



GETTING STARTED WITH TS

To install the TypeScript compiler you need **npm** (Node.js package manager). See section "Tooling" for more information on Node.js and other tools. You can download and install it from here.

Node.js is a open-source and cross-platform JavaScript runtime environment based on the very performant V8 JavaScript engine. With Node.js you can build scalable network applications.

To install the TypeScript compiler `tsc` globally on your machine, run

```
npm install -g typescript
```

in a terminal.



Compiling and Running TypeScript code

TypeScript code is stored in ".ts"-files. Let's assume you have a TS file "foo.ts", then tsc would "transpile" your TS code to JS and save it in "foo.js". This is necessary because browsers as well as Node.js processes only "understand" JS code.

You can also directly execute a TS file with `ts-node`. To install it, run `npm install -g ts-node`.

To run foo.ts you simply execute

```
ts-node ./foo.ts
```

In case your TS code should be executed in a browser, you need the tsc output (JS code). In this section we will use a Node.js/npm project to demonstrate the features of TS.

RUNNING TS CODE WITH NODE.JS

You can initialize your TS project "from scratch" or use a preset project template. We will use the second approach.

Open folder `02-ts/hello-world` from the Git repository with VS Code.

Now open a terminal and run `npm install` to install the required packages (see package.json). Checkout file "README.md" for information on how to run and build your project.

Run

```
npm run dev
```

use your own name in line 6 of "src/hello-world.ts" and see what happens!



Some useful TS Resources:

- TypeScript Home: https://www.typescriptlang.org/
- Tutorial (one of many): https://www.typescripttutorial.net/
- TS Cheatsheets: https://www.typescriptlang.org/cheatsheets/



2.2 TYPES



Preface: Some of the examples assume that you are familiar with functions and arrow functions in JavaScript.

A *type* in a programming language defines properties (attributes) and functions (operations) that a value of that type has. TS supports many kinds of types:

- primitive types
- special types: any, unknown, null, undefined
- arrays
- object types
- union types
- type aliases
- interfaces
- generics
- string literal types
- enum types
- tuple types

PRIMITIVES

TS just like JS supports three main primitve types:

- boolean: values *true* and *false*
- number: whole numbers and floating point numbers
- string: texts like "TypeScript - how cool is that?"

Example:

```
let name = "Rajesh"; // implicit type string
let anotherName: string = "Anna-Maria"; // explicit type
```

In the first line of the example TS infers the type from the value on the left.

SPECIAL TYPES

any

In rare situations you may want to avoid type checking. The "any" type will help you out:

```
let v: any = true;
v = "string";
console.log(Math.round(v)); // output "NaN" (cannot round a string value)
```

- will not produce an error since "v" can be any type
- round() expects a number, but no error will be shown since "v" can be any type

unknown

Just like `any`, `unknown` can be used in situations where you want to bypass type checking. However, with `unknown` the TS inference mechanism will check the type before you use a value of type `unknown`.

```
let result: unknown;
result = [1, 2, 3];

const total = (result as number[]).reduce((a: number, b: number) => a + b, 0
console.log(total); // 6
```

If you remove the type assertion "result as number[]" the compiler will produce the following error:
"error TS2339: Property 'reduce' does not exist on type 'unknown'."



null and undefined

JavaScript has two primitive values used to signal absent or uninitialized value: `null` and `undefined`. In TypeScript it depends on the option "strictNullChecks" what happens. When "strictNullChecks" is `on`, you will need to test for those values before using methods or properties on that value.

Example:

```
function doSomething(x: string | null) {
  if (x !== null) {
    console.log("Hello, " + x.toUpperCase());
  }
}
```

Parameter "x" may be a string or "null". "strictNullChecks==On" will cause a compiler error unless you explicitly check for null.



ARRAYS

Unlike in JS where arrays can contain elements of arbitrary types, arrays in TS need to be type homogeneous, i.e. contain elements of the same type.

To create an array of strings, you type:

```
const stringArray = ["string 1", "string 2"];
const stringArray2 = new Array<string>(); // string array using generics
```

Remember that an array is an object: typeof(stringArray) will return "object".

Since TS is a superset of JS, you can use any array method that you can use in JS:

```
const numbers = [1, 4, 9, 16];
const squares = numbers.map((x) => x * x);
console.log(squares); // output: Array [1, 16, 81, 256]
```

OBJECT TYPES

In TS you can define a type for an object "on the fly", i.e. without referring to an already defined type:

```
let employee: {
  firstName: string;
  lastName: string;
  age: number;
  jobTitle: string;
};
```

Of course, you can define an explicit type instead of using an object type as well (see next pages). In many cases, however, it saves typing. Keep in mind that this only works because of the way TS compares types (duck typing)!

EXERCISE: OBJECT TYPES I

Given an object type for a person with a name and a toString function. Mark the correct definition.

- ☐ [name: string, toString: () => string]
- ☐ { name: string, toString: () => string }
- ☐ { name: string, toString: string }

EXERCISE: OBJECT TYPES II

Define a person named 'Anna' based on the object type from the previous exercise.

- [ ] `const anna: { name: string, toString: () => string } = { name: 'Anna', toString: () => 'Anna' };`
- [ ] `const anna: { name: string, toString: () => string } = { name: 'Anna', toString: () => this.name };`
- [ ] `const anna: { name: string, toString: () => string } = { 'Anna', () => 'Anna' };`

UNION TYPES

Sometimes functions return different types depending on their arguments. Consider the following example:

```
function add(a: any, b: any) {
  if (typeof a === "number" && typeof b === "number") {
    return a + b;
  }
  if (typeof a === "string" && typeof b === "string") {
    return a.concat(b);
  }
  throw new Error("Parameters must be numbers or strings");
}

console.log(add(1, 2)); // output: 3
console.log(add("1", "2")); // output: "12"
```

However, adding `console.log(add(true, false));` would result in an error message "Error: Parameters must be numbers or strings".

Since this is hard to understand for a caller of add() without looking at the implementation, we can make this more explicit using a union type:

```
function add(a: number | string, b: number | string): number | string {
...
```

The type "number | string" means that a value must be either of type number or string to be valid.

The function header above specifies that both arguments must be either of type number or string. The result of the function will also be either of type number or string.

EXERCISE: UNION TYPES

Which of the following types defines a union type for a function which returns either void, a number or an array of numbers? Check all that apply.

- ☐ void | number | Array<number>
- ☐ void | number | number[]
- ☐ void & number & Array<number>

TYPE ALIASES

A type alias lets you define a new type for a - typically more complex - existing type.

Some examples:

```
type Name = string;
let firstName: Name = "Anna";

type Person = {
  name: string;
  age: number;
};
let anna: Person = { name: "Anna", age: 34 };

type alphanumeric = string | number;
let alnum1: alphanumeric = "a string",
  alnum2: alphanumeric = 123.56;
```

- a simple alias for the string type
- Person is an object type with two properties
- alphanumeric is a shorthand for the union type string | number

EXERCISE: TYPE ALIASES

Define a type alias "all" for alphanumeric or boolean values (based on the previous example).

- [ ] type all = alphanumeric | boolean;
- [ ] type all = alphanumeric | "boolean";
- [ ] type all = alphanumeric | "true" | "false";

Intersection Types

An intersection type is a type that combines the properties of multiple existing types. In the following example we define a type alias for the intersection type `Personal & Contact`:

```
type Personal = { name: string; age: number };
type Contact = { email: string; phone: string };
type Candidate = Personal & Contact;
let candidate: Candidate = {
  name: "Joe",
  age: 25,
  email: "joe@example.com",
  phone: "(408)-123-4567",
};
```

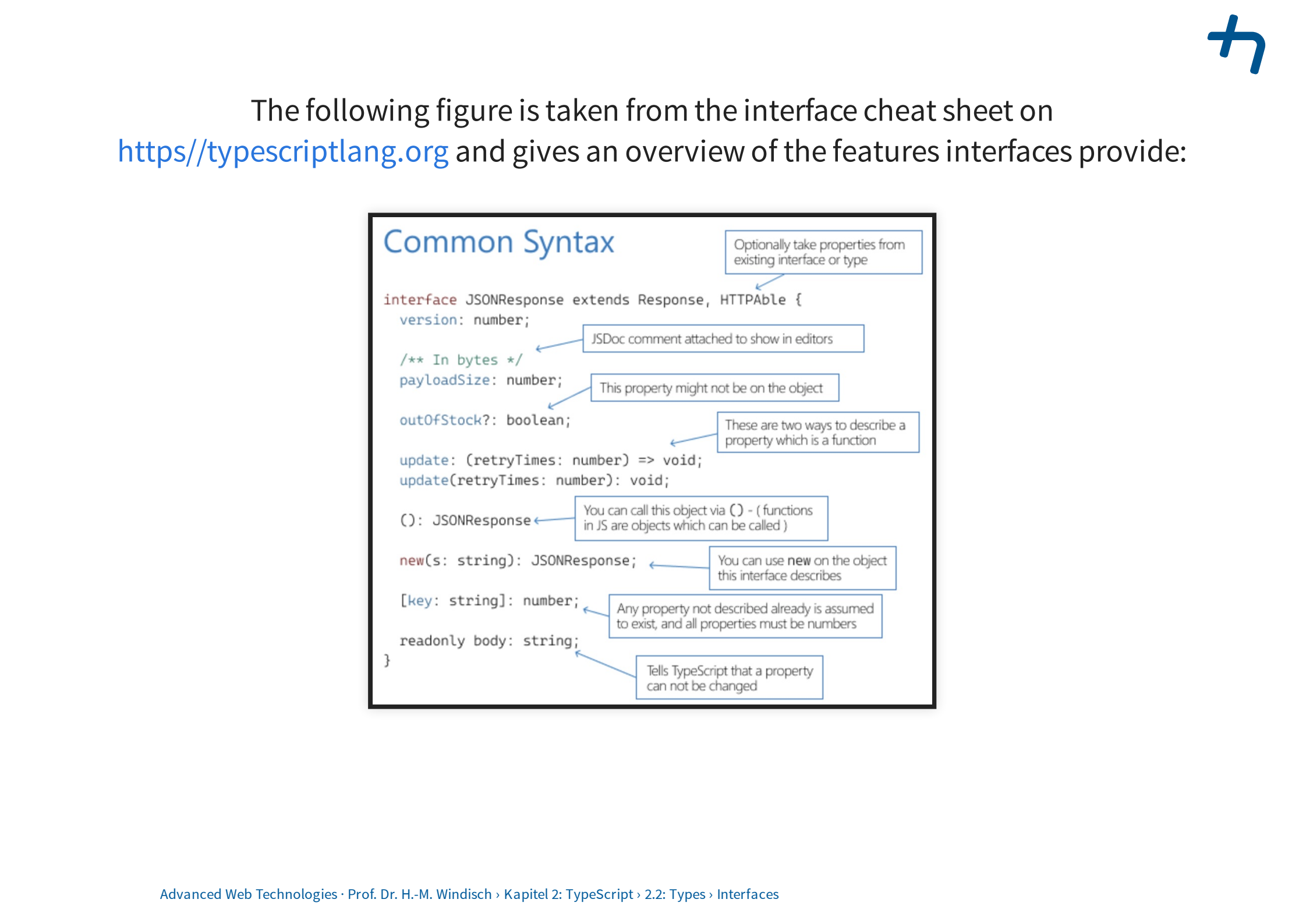
INTERFACES

An interface defines the shape of an object, i.e. its properties and methods. Just like in Java, interfaces are often used to enforce consistency across classes or objects.

- Classes typically `implement` one or more interfaces.
- An interface may also `extend` other interfaces which means it inherits the properties of each extended interface.

Example:

```
interface Person {
  name: string;
  age: number;
  speak(): void;
}
```



The following figure is taken from the interface cheat sheet on https//typescriptlang.org and gives an overview of the features interfaces provide:

EXERCISE: INTERFACES I

Given an interface:

```
interface Order {
  id: string;
  billingDate: Date;
}
```

How can you supply the missing function for billing the order (and setting the billing date)? Check all that apply.

EXERCISE: INTERFACES I

- [ ] bill(d: Date): void
- [ ] bill: (d: Date) => void

EXERCISE: INTERFACES II - LIBRARY SYSTEM

Imagine you are creating a library system to manage books, members, and loans. You'll use TypeScript interfaces to define the structure of these entities.

Step 1: Define Interfaces

1. Create an interface called `Book` with the following properties:
   - `title`: The title of the book.
   - `author`: The author of the book.
   - `isbn`: The ISBN number of the book.
   - `isAvailable`: Indicates if the book is currently available for borrowing.

2. Create an interface called `Member` with the following properties:

   - `id`: A unique identifier for the member.
   - `name`: The name of the member.
   - `email`: The email address of the member.
   - `borrowedBooks`: A list of books currently borrowed by the member.

3. Create an interface called `Loan` with the following properties:

   - `book`: The book that is being borrowed.
   - `member`: The member who borrowed the book.
   - `loanDate`: The date the book was borrowed.
   - `returnDate` (Date or `null`): The date the book was returned, or `null` if it hasn't been returned yet.

Step 2: Use the Interfaces

1. Create a few example objects for each interface (e.g., three books, two members, and one loan).

2. Write a function `borrowBook` that takes a `Member` and a `Book` as parameters and:

    - Checks if the book is available.
    - If it is, adds the book to the member's `borrowedBooks` array and marks the book as unavailable.
    - If it is not, logs that the book is already borrowed.

3. Write a function `returnBook` that takes a `Member` and a `Book` and:

    - Removes the book from the member's `borrowedBooks` array.
    - Marks the book as available.

GENERICS

Basically, a generic type is a type that has one or more parameters which are types.
You can define generic functions, classes and interfaces.

Example of a generic function:

```
function getRandomElement<T>(items: T[]): T {
  let randomIndex = Math.floor(Math.random() * items.length);
  return items[randomIndex];
}
// usage: printing a random element of a number array and a string array
let numbers = [1, 5, 7, 4, 2, 9];
console.log(getRandomElement<number>(numbers));
let colors = ["red", "green", "blue"];
console.log(getRandomElement(colors));
```

- T is the type parameter
- T is set to "number", so the compiler can check the function's parameter type
- here T is automatically inferred by the compiler from the function argument. T will implicitely be set to "string"

Example of a generic Interface

```
interface Pair<T, U> {
  first: T;
  second: U;
}

let numberPair: Pair<number, number> = { first: 1, second: 2 };
let stringNumberPair: Pair<string, number> = { first: "Age", second: 30 };
```

- Pair has two parameters for the first and the second property
- usage examples with different type pairs

TYPE ASSERTIONS

Type assertions in TypeScript are a way for developers to explicitly tell the TypeScript compiler that they know the type of a variable better than the compiler does. TypeScript uses type inference, but sometimes the compiler's inference may be too general or incorrect for a specific case. Type assertions allow you to override the inferred type with a more specific type.

Example:

```
let inputElement = document.getElementById("user-input");
(inputElement as HTMLInputElement).value = "Hello";
```

- inferred as HTMLElement | null
- Type assertion to treat it as an HTMLInputElement

EXERCISE: GENERICS

Given the following class for a Wrapper for numbers:

```
class Wrapper {
  private content: number;

  constructor(value: number) {
    this.content = value;
  }
  getContent(): number {
    return this.content;
  }
}
// usage of generic version (not working yet!)
let numberWrapper = new Wrapper<number>(123);
let stringWrapper = new Wrapper<string>("Hello");
console.log(numberWrapper.getContent()); // Output: 123
console.log(stringWrapper.getContent()); // Output: Hello
```

EXERCISE: CLASS WRAPPER EXERCISE

Which changes do you need to apply to class Wrapper to make the code work?

- [ ] "content: T;" instead of "content: number;"
- [ ] "class Wrapper<T>" instead of "class Wrapper"
- [ ] "return this.content as T;" instead of "return this.content;"
- [ ] "getContent(): T {" instead if "getContent(): number {"
- [ ] "constructor(value: T) {" instead of "constructor(value: number) {"

STRING LITERAL TYPES

The string literal types allow you to define a type that accepts only one specified string literal. The following defines a string literal type that accepts a literal string 'click':

```
let click: "click"; // define variable click of type 'click'
click = "click"; // ok
click = "abc"; // compiler error
```

You might ask yourself, what string literal types are good for...



Imagine you want to handle different types of mouse events: click, double click, mouse up and mouse down. A type system that might support you in this situation would look like this:

```
type MyMouseEvent = "click" | "dblclick" | "mouseup" | "mousedown";
let mouseEvent: MyMouseEvent;

mouseEvent = "click"; // valid
mouseEvent = "dblclick"; // valid
mouseEvent = "mouseup"; // valid
mouseEvent = "mousedown"; // valid
mouseEvent = "mouseover"; // compiler error
```

Of course, you could also define a enum type which will be discussed next.

ENUM TYPES

In cases where you have a small number of related constant values that are known at compile time, you can use an enum.

Example:

```
enum ApprovalStatus {
  draft = 1,
  submitted,
  approved,
  rejected,
}
```

- ApprovalStatus has 4 fixed values
- internal numbering of enum values typically starts with "0", here we make it start with "1" (submitted will be assigned "2" etc.)

Using our example enum ApprovalStatus:

```
const request = {
  id: 1,
  status: ApprovalStatus.approved,
  description: "Please approve this request",
};

if (request.status === ApprovalStatus.approved) {
  // send an email
  console.log("Send email to the Applicant...");
}
```

TUPLE TYPES

A tuple can be considered as an array with a fixed length and fixed element types.

Example: Let's define a type for RGB values

```
type RGBValue = [number, number, number];
let color: RGBValue = [255, 0, 0];
```

- RGBValue is an alias for the tuple type
- the compiler checks if the given array values comply to the element types given in the tuple type and if the number of elements is correct



2.3 FUNCTIONS

Functions in TS look like JS functions with the only difference being that you add parameter types and a return type.

Example of a function with a default parameter:

```
function applyDiscount(price: number, discount: number = 0.05): number {
  return price * (1 - discount);
}
console.log(applyDiscount(100));          // output: 95
console.log(applyDiscount(100, 0.1));     // output: 90
```

- "discount" has a default value of 0.05 if it is not specified
- here "discount" is specified, so the actual parameter value is used

OPTIONAL PARAMETERS

Optional parameters must be specified after mandatory and default parameters:

```
function applyDiscount(price: number, discount?: number): number {
  if (!discount)
    discount = 0.05;
  return price * (1 - discount);
}
console.log(applyDiscount(100));          // output: 95
console.log(applyDiscount(100, 0.1));     // output: 90
```

- if "discount" is not specified, the parameter value in "applyDiscount" is `undefined`

REST PARAMETERS

A rest parameter allows a function to accept zero or more arguments of the specified type. A function can only have one rest parameter of type "Array" which must appear last in the parameter list.

Example: creating a map from multiple values of type T

```
function createMap<T>(...elements: T[]): Map<number, T> {
  const m = new Map<number, T>();
  elements.forEach((value: T, index: number) =>  m.set(index, value));
  return m;
}

console.log(createMap(1, 2, 3)); // Map(3) { 0 => 1, 1 => 2, 2 => 3 }
console.log(createMap("apple", "banana", "cherry"));
// Map(3) { 0 => 'apple', 1 => 'banana', 2 => 'cherry' }
```

FUNCTION TYPES

When a function has a function as a parameter you need to specify a function type for that parameter. The following example shows how to do that:

```
function calculate(value1: number, value2: number,
    operator: (v1: number, v2: number) => number): number {
  return operator(value1, value2);
}

console.log(calculate(2, 10, (x, y) => x + y)); // 12
console.log(calculate(2, 10, (x, y) => x * y)); // 20
```

- calculate returns the result of the calculation performed by operator based on the two values
- operator is a parameter of a function type. It receives two number parameters and returns a number
- we apply operator to the values and return the result
- here we call the calculate function with different arrow functions to perform different computations

FUNCTION OVERLOADING

Function overloading allows you to define multiple signatures for a single function and provide one implementation that handles all defined signatures.

Example: array conversion

Step 1: Define signatures

```
function toArray(item: string): string[];
function toArray(item: number): number[];
function toArray(item: string[]): string[];
function toArray(item: number[]): number[];
```



Step 2: implement function and use it

```
function toArray(item: any): string[] | number[] {
    return Array.isArray(item) ? item : [ item ];
}
// use function
console.log(toArray(42));          // Output: [42]
console.log(toArray("hello"));     // Output: ["hello"]
console.log(toArray([1, 2, 3])); // Output: [1, 2, 3]
console.log(toArray(["a", "b"]));// Output: ["a", "b"]
```

- if the parameter is already an array we just return it, otherwise we create a new array with the parameter value as only element
- note that function overloading in TS is unlike the function overloading supported by other statically typed languages such as Java



2.4 CLASSES

The following example shows you how to define a TS class:

Part I: Class Header & fields

```
class User
    extends Account                          // User is subclass of Account
    implements Updatable, Serializable {  // user conforms to 2 interfaces

  id: string;                          // a public field
  displayName?: boolean;               // an optional field, my be undefined
  name!: string;                       // name is always defined!
  #attributes: Map<string, any>;       // a private field
  roles = [ "user" ];                  // a public field with a default value
  readOnly createdAt = new Date(); // a readonly public field
}
```

Part II: Constructor & getter/setter

```
...
constructor(id: string,
    public age: number, // age is public field, initialized in CTOR
    email: number) {
  super(id);
  this.email = email; // if "strict: true": will cause error
}

get id() {              // getter for id
  return this.id;
}
set id(value: string) { // setter for id
  this.id = value;
}
...
```

- a shortcut for defining a field and initializing it in the CTOR

Part III: Methods and Static Elements

```
  ...
  setName(name: string) { // public method
    this.name = name;
  }
  private getAge() { // not usable from outside or subclasses
    return this.age;
  }
  protected setAge(value: number) { // may be used in subclasses
    this.age = value;
  }

  static #userCount = 0;                    // static private field
  static registerUser(user: User) { ... } // static public method

  static { this.#userCount = 1; } // initialization of static fields
}
```

- a shortcut for defining a field and initializing it in the CTOR

ABSTRACT CLASSES

As with Java, TS allows you to define an abstract class that cannot be instantiated. Just like in Java the abstract class may contain abstract methods without implementation.

Example:

```
abstract class Employee {
  constructor(private firstName: string, private lastName: string) {}
  abstract getSalary(): number;
  get fullName(): string {
    return `${this.firstName} ${this.lastName}`;
  }
  compensationStatement(): string {
    return `${this.fullName} makes ${this.getSalary()} a month.`;
  }
}
```

- define private attributes "firstName" and "lastName"
- "getSalary()" must be implemented by non-abstract subclasses

2.5 CONTROL FLOW ANALYSIS IN TS

The fundamental control flow structures (like if, for, while, etc.) in TS are the same as in JS.

However, TypeScript introduces a feature called control flow analysis that enhances the type system and enables stricter type-checking based on the flow of your code.

Example

```
function printId(id: number | string) {
  if (typeof id === "string") {
    console.log(id.toUpperCase());
  } else {
    console.log(id);
  }
}
```

- in this branch, id is of type 'string', so the method "toUpperCase()" is valid. Without analysis, the compiler would not know whether "id" is number (=> error) or string (ok)
- in this branch, id is of type 'number'

2.6 USEFUL OPERATORS

Following we explain some useful operators that can help you write more compact code.

SPREAD OPERATOR (TS AND JS)

The "..." operator expands elements of an iterable value. Iterables in TS (and JS) are: Arrays, Strings, Maps, Sets and custom iterables that implement the Iterable interface.

Examples: copy arrays, merge objects

```
// copy array
const source = [1, 2, 3];
console.log([...source]);

// merge objects
const obj1 = { value: 12.45, unit: 'km/h' };
const obj2 = { date: new Date(), unit: 'm/s' };
console.log({ ...obj1, ...obj2 }); // obj2.unit overwrites obj1.unit!
```

Examples: pass array to function, convert string to array

```
function addNumbers(a: number, b: number, c: number): number {
    return a + b + c;
}
const numbers = [1, 2, 3];
console.log(addNumbers(...numbers)); // Output: 6

const greeting = "Hello";
const characters = [...greeting];
console.log(characters); // Output: ['H', 'e', 'l', 'l', 'o']
```

Please note that characters is of type string[]!

OPTIONAL CHAINING OPERATOR

In situations where you access deeply nested properties you typically need to check every property before accessing the next. The optional chaining operator eliminates the need for those checks. If a property is `undefined`, the whole expressions will yield `undefined`.

Example:

```
type Address = { street?: string; city?: string; };
type User = { name: string; address?: Address; };
const user: User = {
    name: "Alice",
    // address is missing here, which would normally
    // cause an error when accessing nested properties
};
console.log(user?.address?.city); // Output: undefined
```

DESCTRUCTURING (TS AND JS)

Destructuring is a feature in TypeScript (and JavaScript) that allows you to unpack values from arrays or properties from objects into distinct variables.

Destrucuring for arrays:

```
const numbers = [1, 2, 3, 4];
// Destructuring array elements into variables
const [first, second, third] = numbers;
console.log(first);  // Output: 1
console.log(second); // Output: 2
console.log(third);  // Output: 3

// Skipping the second element
const [first2, , third2] = numbers;
console.log(first2);  // Output: 1
console.log(third2);  // Output: 3

// Using the rest parameter to capture remaining elements
const [first3, ...rest] = numbers;
console.log(first); // Output: 1
console.log(rest);  // Output: [2, 3, 4, 5]
```

ASSERTION OPERATOR

In TypeScript, the ! operator is called the non-null assertion operator. It is used to tell the TypeScript compiler that you are certain a value is not null or undefined, even if TypeScript's type-checker isn't able to verify it.

Example:

```
let element = document.getElementById("myElement");
element!.style.backgroundColor = "red";
```

"element!" asserts that "element" is not null or undefined, allowing you to access its properties or methods without the compiler raising a potential null or undefined error.

Please use this operator with caution!

2.7 DECORATORS

Decorators in TypeScript provide a way to add annotations and a meta-programming syntax for class declarations and members. They are a special kind of declaration that can be attached to a class, method, accessor, property, or parameter.

A SIMPLE CLASS DECORATOR

The following example will add a function "greet()" to a class.

```
function Greeter(target: Function) {
    target.prototype.greet = function () {
        console.log("Hello from the decorator!");
    };
}

@Greeter
class Person {
    name: string;
    constructor(name: string) { this.name = name; }
}
// usage
const person = new Person("Alice");
(person as any).greet(); // Output: Hello from the decorator!
```

- the decorator is applied to class Person
- when the decorator is processed, the class description will be extended by function "greet()"
- person does not have a greet() function (as per the class definition), so we need to cast it to "any"

METHOD DECORATOR EXAMPLE

```
function logExecutionTime(originalMethod: any, _context: any) {
  function replacementMethod(this: any, ...args: any[]) {
    console.time(originalMethod.name); // Start timing
    const result = originalMethod.call(this, ...args);
    console.timeEnd(originalMethod.name); // End timing
    return result;
  }
  return replacementMethod;
}
```

- define decorator "logExecutionTime". It will be passed the function that is being called
- define replacement method which surrounds the actual function call with timing measurements
- call the original function with the args from the original function call
- return the replacement method, it will be called instead of the original function

```
class MathOperations {
  @logExecutionTime
  calculateSquare(n: number): number {
    // Simulate a time-consuming task
    for (let i = 0; i < 1_000_000_000; i++) {}
    return n * n;
  }
}

// call class method
const math = new MathOperations();
console.log(`Result: ${math.calculateSquare(3)}`);
```

- annotate "calculateSquare" with decarator
- when "calculateSquare" is called, the replacement method of the decorator is called...



2.8 MODULES



When developing a bigger application, modularization through folders and files is a necessity. To support modularization, both JavaScript and TypeScript offer several module concepts.

A module is a file containing variables, functions and classes. Everything in the module is automatically private - i.e cannot be used from outside. Only those elements that are explicitly **exported** may be visible to other modules.

If a module wants to use an exported element from another module, it must **import** the element before using it.



PRIMARY TYPESCRIPT MODULE SYSTEMS

Following the two main module systems are briefly characterized. They both support ".js" and ".ts" files.

ES Modules (ECMAScript Modules):

- import through `import` keyword, export through `export` keyword
- supported in most modern JavaScript environments (e.g., browsers, Node.js)

CommonJS Modules: typically used in ".js" files

- import through `require`
- export through `module.exports` or `exports`
- used primarily in Node.js applications

We will focus on ES modules here.

EXPORTING AND IMPORTING

Let's assume we want to provide a module "mathUtils.ts":

```
export function add(a: number, b: number): number {
    return a + b;
}
export function subtract(a: number, b: number): number {
    return a - b;
}
// or: export { add, subtract };
```

The module can be used like this, e.g. in "main.ts":

```
import { add, subtract as sub } from './mathUtils';

console.log(add(5, 3)); // Output: 8
console.log(sub(5, 3)); // Output: 2
```

- an imported element may be aliased. Here: "sub" as alias for "subtract"

DEFAULT EXPORTS

When a module is exporting a single primary value or object, it may have a single default export.

Example: module "greeter.ts"

```
export default function greet(name: string): void {
  console.log(`Hello, ${name}!`);
}
```

Usage in "main.ts":

```
// main.ts
import greet from './greeter';

greet("Alice"); // Output: Hello, Alice!
```

Without the default export you would have to write the import statement in a different way: "import { greet } from 'greeter'".

TYPE DECLARATIONS

Some third party libraries do not provide TS type declarations. In that case you might provide a declaration file yourself.

Example: Typing file "mathUtils.d.ts"

```
declare module 'mathUtils' {
    export function add(a: number, b: number): number;
    export function subtract(a: number, b: number): number;
}
```



